

BIG DATA ANALYTICS LAB MANUAL

EXPERIMENTS 7–10

Format: Aim → Algorithm / Procedure → Program / Commands → Output → Result

EXPERIMENT 7 — Pig Latin Filtering, Grouping, Joining and Aggregation

AIM

To perform Pig Latin filtering, grouping, joining and aggregation.

ALGORITHM / PROCEDURE

1. Load the student dataset.
2. Filter records by marks.
3. Group records by department.
4. Calculate department-wise averages and counts.
5. Load the department dataset.
6. Join the relations.
7. Display results using DUMP.

PROGRAM / COMMANDS

Start Pig

```
pig -x local
```

Load student dataset

```
students = LOAD 'students.csv' USING PigStorage(',')
AS (id:int,name:chararray,dept:chararray,marks:int);
```

Filter

```
high_marks = FILTER students BY marks >= 80;
DUMP high_marks;
```

Group and average

```
dept_group = GROUP students BY dept;
dept_avg = FOREACH dept_group
GENERATE group AS dept, AVG(students.marks) AS average_marks;
DUMP dept_avg;
```

Count

```
dept_count = FOREACH dept_group
GENERATE group AS dept, COUNT(students) AS student_count;
DUMP dept_count;
```

Load departments

```
departments = LOAD 'departments.csv' USING PigStorage(',')
AS (dept:chararray,hod:chararray);
```

Join

```
joined = JOIN students BY dept, departments BY dept;
DUMP joined;
```

Exit

```
quit;
```

OUTPUT

The output displays filtered students, department-wise average marks, department-wise student counts, and joined records.

RESULT

Thus, Pig Latin filtering, grouping, joining and aggregation were successfully demonstrated.

EXPERIMENT 8 — Hive Tables and HiveQL

AIM

To create Hive tables and execute HiveQL queries.

ALGORITHM / PROCEDURE

1. Start Hadoop and YARN.
2. Start Hive Metastore if it is not already running.
3. Start HiveServer2 if it is not already running.
4. Connect using Beeline.
5. Create studentdb and the student table.
6. Insert, retrieve, filter, group, aggregate and sort records.

PROGRAM / COMMANDS

Start Hadoop

```
start-dfs.sh
start-yarn.sh
jps
```

Metastore — only if not running

```
hive --service metastore &
ss -ltnp | grep 9083
```

HiveServer2 — only if not running

```
hiveserver2 &
ss -ltnp | grep 10000
```

Beeline

```
beeline -u jdbc:hive2://localhost:10000
```

Create database and table

```
CREATE DATABASE studentdb;
USE studentdb;
```

```
CREATE TABLE student(
    student_id INT,
    name STRING,
    department STRING,
    year INT,
    marks INT
);
```

Insert records

```
INSERT INTO student VALUES
(101, 'Arun', 'CSE', 3, 82),
(102, 'Priya', 'AIML', 3, 94),
(103, 'Karthik', 'CSE', 3, 76),
(104, 'Divya', 'AIML', 3, 88),
(105, 'Meena', 'CSE', 3, 95),
(106, 'Ravi', 'IT', 3, 79);
```

Retrieve

```
SELECT * FROM student;
```

Filter

```
SELECT * FROM student WHERE marks >= 80;
```

Group and aggregate

```
SELECT department, COUNT(*) AS student_count, AVG(marks) AS average_marks  
FROM student GROUP BY department;
```

Aggregate and sort

```
SELECT MAX(marks), MIN(marks), AVG(marks) FROM student;  
SELECT * FROM student ORDER BY marks DESC;
```

OUTPUT

Hive displays the database and table, student records, filtered records, department-wise statistics, aggregate values and sorted results after successful execution.

RESULT

Thus, Hive DDL and HiveQL analytical operations were successfully demonstrated.

EXPERIMENT 9 — Data Visualization Using Python

AIM

To create Line, Bar, Pie, Scatter and Box plots using Python libraries.

ALGORITHM / PROCEDURE

1. Import Pandas and Matplotlib.
2. Create the sample dataset.
3. Generate line, bar, pie, scatter and box plots.
4. Display all visualizations.

PROGRAM / COMMANDS

Install libraries

```
pip install pandas matplotlib
```

Program

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May'],
    'Sales': [120, 150, 180, 160, 210],
    'Profit': [20, 30, 42, 35, 50]
})

plt.figure()
plt.plot(df['Month'], df['Sales'], marker='o')
plt.title('Monthly Sales')
plt.xlabel('Month'); plt.ylabel('Sales')
plt.show()

plt.figure()
plt.bar(df['Month'], df['Sales'])
plt.title('Sales by Month')
plt.show()

plt.figure()
plt.pie(df['Sales'], labels=df['Month'], autopct='%1.1f%%')
plt.title('Sales Distribution')
plt.show()

plt.figure()
plt.scatter(df['Sales'], df['Profit'])
plt.title('Sales vs Profit')
plt.xlabel('Sales'); plt.ylabel('Profit')
plt.show()

plt.figure()
plt.boxplot(df['Sales'])
plt.title('Sales Distribution')
plt.show()
```

Run

```
python visualization.py
```

OUTPUT

Five visualizations are displayed: Line Chart, Bar Chart, Pie Chart, Scatter Plot and Box Plot.

RESULT

Thus, data visualization using Python was successfully implemented.

EXPERIMENT 10 — Hive Case Study Using a Sample Dataset

AIM

To perform a Hive case study involving database creation, table creation, dataset loading, retrieval, filtering, grouping, aggregation and sorting.

ALGORITHM / PROCEDURE

1. Create a sample sales CSV file.
2. Upload it to HDFS.
3. Create salesdb.
4. Create the sales table.
5. Load the dataset.
6. Retrieve and filter records.
7. Group and aggregate data.
8. Sort and analyze results.

PROGRAM / COMMANDS

sales.csv

```
1,Arun,Electronics,50000
2,Priya,Clothing,25000
3,Karthik,Electronics,45000
4,Divya,Food,18000
5,Meena,Clothing,32000
6,Ravi,Food,22000
7,Arun,Electronics,55000
8,Priya,Food,28000
```

Upload to HDFS

```
hdfs dfs -mkdir -p /user/hive/input
hdfs dfs -put -f sales.csv /user/hive/input/
```

Connect

```
beeline -u jdbc:hive2://localhost:10000
```

Create database/table

```
CREATE DATABASE salesdb;
USE salesdb;

CREATE TABLE sales(
    id INT,
    customer STRING,
    category STRING,
    amount DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Load

```
LOAD DATA INPATH '/user/hive/input/sales.csv' INTO TABLE sales;
```

Retrieve

```
SELECT * FROM sales;
```

Filter

```
SELECT * FROM sales WHERE amount > 30000;
```

Group and aggregate

```
SELECT category, COUNT(*) AS total_records,  
SUM(amount) AS total_sales,  
AVG(amount) AS average_sales,  
MAX(amount) AS highest_sale,  
MIN(amount) AS lowest_sale  
FROM sales GROUP BY category;
```

Sort

```
SELECT customer, category, amount FROM sales ORDER BY amount DESC;
```

OUTPUT

Workflow output: Create database → Create table → Load dataset → Retrieve records → Filter records → Group records → Aggregate data → Sort and analyze results. For the sample sales data, total records = 8 and total sales = 275000.

RESULT

Thus, the Hive case study workflow was successfully designed and documented using a sample sales dataset.